11. 실제 구현 분석

11.1 코드베이스 구조 분석

Narwhal 프로젝트의 실제 구현을 분석하여 이론과 실제 구현 간의 차이점과 실용적 고려사항을 살펴보겠습니다.

11.1.1 모듈 의존성 분석

```
// Cargo.toml에서 확인된 주요 의존성들

[dependencies]

arc-swap = { version = "1.5.1", features = ["serde"] } // 원자적 포인터 교체

bincode = "1.3.3"  // 직렬화

blake2 = "0.9"  // 해시 함수

bytes = "1.2.1"  // 바이트 버퍼

tokio = { version = "1.20.1", features = ["sync"] } // 비동기 런타임

tracing = "0.1.36"  // 로깅

fastcrypto = "0.1.2"  // 암호화 라이브러리

prometheus = "0.13.2"  // 메트릭 수집
```

11.1.2 실제 구현된 주요 구조체들

```
// consensus/src/consensus.rs에서 실제 구현
```

```
pub struct ConsensusState {
pub last_committed_round: Round,
pub last_committed: HashMap<PublicKey, Round>,
pub dag: Dag,
pub metrics: Arc<ConsensusMetrics>,
}


// 실제 DAG 타입 정의
pub type Dag = HashMap<Round, HashMap<PublicKey, (CertificateDigest, Certificat


// 컨센서스 출력 형식
#[derive(Serialize, Deserialize, Clone, Debug)]
pub struct ConsensusOutput {
pub certificate: Certificate,
pub consensus_index: SequenceNumber,
}
```

11.2 Tusk vs Bullshark 구현 비교

11.2.1 핵심 차이점 실제 코드 분석

```
// Tusk 구현 (consensus/src/tusk.rs)
impl ConsensusProtocol for Tusk {
```

```rust
fn process_certificate(
&mut self,
state: &mut ConsensusState,
consensus_index: SequenceNumber,
certificate: Certificate,
) -> StoreResult<Vec<ConsensusOutput>> {
let round = certificate.round();
let r = round - 1;


// Tusk: 4라운드 이상에서만 처리, r-2 라운드 리더 확인
if r % 2 != 0 || r < 4 {
return Ok(Vec::new());
}


let leader_round = r - 2; // 핵심 차이점 1
// r-1 라운드에서 지지 확인
let stake: Stake = state
.dag
.get(&(r - 1)) // 핵심 차이점 2
.expect("We should have the whole history by now")
.values()
.filter(|(_, x)| x.header.parents.contains(leader_digest))
.map(|(_, x)| self.committee.stake(&x.origin()))
```

```
.sum();
}
}

// Bullshark 구현 (consensus/src/bullshark.rs)
impl ConsensusProtocol for Bullshark {
fn process_certificate(
&mut self,
state: &mut ConsensusState,
consensus_index: SequenceNumber,
certificate: Certificate,
) -> StoreResult<Vec<ConsensusOutput>> {
let round = certificate.round();
let r = round - 1;

// Bullshark: 2라운드 이상에서만 처리, r 라운드 리더 확인
if r % 2 != 0 || r < 2 { // 더 빠른 시작
return Ok(Vec::new());
}

let leader_round = r; // 즉시 리더 확인

// 현재 라운드에서 지지 확인
```

```
let  stake: Stake = state

.dag

.get(&round) // 현재 라운드에서 확인

.expect("We should have the whole history by now")

.values()

.filter(|(_, x)| x.header.parents.contains(leader_digest))

.map(|(_, x)| self.committee.stake(&x.origin()))

.sum();

}

}
```

11.2.2 리더 선출 구현

```
// 두 프로토콜 모두 동일한 리더 선출 로직 사용

impl Tusk {

fn  leader<'a>(

committee: &Committee,

round: Round,

dag: &'a Dag,

) -> Option<&'a (CertificateDigest, Certificate)> {

cfg_if::cfg_if! {

if #[cfg(test)] {

// 테스트에서는 첫 번째 노드를 리더로 사용
```

```
let  leader = committee.authorities.iter().next()
.expect("Empty authorities table!").0;
} else {
// 실제 환경에서는 스테이크 가중 선출
let  leader = &committee.leader(round);
}
}

dag.get(&round).and_then(|x| x.get(leader))
}
}
```

11.3 DAG 구현 세부사항

11.3.1 실제 DAG 관리 구현

```
// consensus/src/dag.rs에서 실제 DAG 구현
struct InnerDag {
rx_primary: metered_channel::Receiver<Certificate>,
rx_commands: Receiver<DagCommand>,
dag: NodeDag<Certificate>, // 실제 DAG 데이터 구조
vertices: RwLock<BTreeMap<(PublicKey, Round), CertificateDigest>>,
metrics: Arc<ConsensusMetrics>,
```

```
}


// DAG 명령어 타입들
#[allow(clippy::large_enum_variant)]
enum DagCommand {
Insert(Box<Certificate>, oneshot::Sender<Result<(), ValidatorDagError>>),
Contains(CertificateDigest, oneshot::Sender<bool>),
HasEverContained(CertificateDigest, oneshot::Sender<bool>),
Rounds(PublicKey, oneshot::Sender<Result<RangeInclusive<Round>, ValidatorDagErro
ReadCausal(CertificateDigest, oneshot::Sender<Result<Vec<CertificateDigest>, Val
NodeReadCausal((PublicKey, Round), oneshot::Sender<Result<Vec<CertificateDigest>
Remove(Vec<CertificateDigest>, oneshot::Sender<Result<(), ValidatorDagError>>),
NotifyRead(CertificateDigest, oneshot::Sender<Result<Certificate, ValidatorDagEr
}
```

11.3.2 메모리 최적화 실제 구현

```
impl InnerDag {
#[instrument(level = "trace", skip_all, fields(certificate = ?certificate), err)
fn  insert(&mut  self, certificate: Certificate) -> Result<(), ValidatorDagError
let  digest = certificate.digest();
let  round = certificate.round();
let  origin = certificate.origin();
```

```
{
// 락 기반 원자성 보장
let mut vertices = self.vertices.write().unwrap();
// 실제 DAG에 삽입 시도
self.dag.try_insert(certificate)?;
// 보조 인덱스 업데이트
vertices.insert((origin, round), digest);
}

// 메트릭 업데이트
self.update_metrics();

Ok(())
}

fn update_metrics(&self) {
let vertices = self.vertices.read().unwrap();

self.metrics
.external_consensus_dag_vertices_elements
.with_label_values(&[])
.set(vertices.len() as i64);
```

```
self.metrics
.external_consensus_dag_size
.with_label_values(&[])
.set(self.dag.size() as i64)
}
}
```

11.4 상태 관리 실제 구현

11.4.1 ConsensusState 업데이트 로직

```
impl ConsensusState {
pub fn update(&mut self, certificate: &Certificate, gc_depth: Round) {
// 최신 커밋 라운드 추적
self.last_committed
.entry(certificate.origin())
.and_modify(|r| *r = max(*r, certificate.round()))
.or_insert_with(|| certificate.round());

let last_committed_round = *std::iter::Iterator::max(self.last_committed.values
self.last_committed_round = last_committed_round;
```

```
// 메트릭 업데이트
self.metrics
.last_committed_round
.with_label_values(&[])
.set(last_committed_round  as i64);


// 가비지 컬렉션: GC 깊이를 초과한 라운드 제거
self.dag.retain(|r, _| r + gc_depth >= last_committed_round);
// 개별 노드의 오래된 인증서 제거
for (name, round) in &self.last_committed {
self.dag.retain(|r, authorities| {
if  r < round {
authorities.remove(name);
}
!authorities.is_empty()
});
}
}
}
```

11.4.2 저장소에서 상태 복구

```rust
impl ConsensusState {
#[instrument(level = "info", skip_all)]
pub  async  fn  new_from_store(
genesis: Vec<Certificate>,
metrics: Arc<ConsensusMetrics>,
recover_last_committed: HashMap<PublicKey, Round>,
cert_store: CertificateStore,
gc_depth: Round,
) -> Self {
let  last_committed_round = *recover_last_committed
.iter()
.max_by(|a, b| a.1.cmp(b.1))
.map(|(_k, v)| v)
.unwrap_or_else(|| &0);


if  last_committed_round == 0 {
return  Self::new(genesis, metrics);
}
metrics.recovered_consensus_state.inc();


// 저장소에서 DAG 재구성
let  dag = Self::construct_dag_from_cert_store(
```

```
cert_store,
last_committed_round,
gc_depth
).await;


Self {
last_committed_round,
last_committed: recover_last_committed,
dag,
metrics,
}
}


#[instrument(level = "info", skip_all)]
pub  async  fn  construct_dag_from_cert_store(
cert_store: CertificateStore,
last_committed_round: Round,
gc_depth: Round,
) -> Dag {
let  mut  dag: Dag = HashMap::new();
info!("Recreating dag from last committed round: {}", last_committed_round);


let  min_round = last_committed_round.saturating_sub(gc_depth);
```

```
// 최소 라운드 이후의 모든 인증서 로드
let  cert_map = cert_store.after_round(min_round + 1).unwrap();


let  num_certs = cert_map.len();
for (digest, cert) in  cert_map.into_iter().map(|c| (c.digest(), c)) {
let  inner = dag.get_mut(&cert.header.round);
match  inner {
Some(m) => {
m.insert(cert.header.author.clone(), (digest, cert.clone()));
}
None => {
dag.entry(cert.header.round)
.or_insert_with(HashMap::new)
.insert(cert.header.author.clone(), (digest, cert.clone()));
}
}
}
info!("Dag was restored and contains {} certs for {} rounds", num_certs, dag.len


dag
}
}
```

11.5 성능 최적화 실제 구현

11.5.1 메트릭 수집 최적화

```
impl Consensus<Protocol> {
async fn run(&mut self, /* ... */) -> StoreResult<()> {
loop {
tokio::select! {
Some(certificate) = self.rx_primary.recv() => {
// 인증서 처리
let sequence = self.protocol
.process_certificate(&mut state, self.consensus_index, certificate)?;

self.consensus_index += sequence.len() as u64;

for output in sequence {
let certificate = &output.certificate;
// 벤치마크 모드에서만 상세 로깅
#[cfg(feature = "benchmark")]
for digest in certificate.header.payload.keys() {
tracing::info!("Committed {} -> {:?}", certificate.header, digest);
}
```

```
// 주기적으로만 DAG 크기 메트릭 업데이트 (성능 최적화)
if  output.consensus_index % 1_000 == 0 {
self.metrics.dag_size_bytes.set(
(mysten_util_mem::malloc_size(&state.dag) +
std::mem::size_of::<Dag>()) as i64
);
}

// Primary에게 인증서 전송
self.tx_primary.send(certificate.clone()).await
.expect("Failed to send certificate to primary");

// 애플리케이션 레이어에 출력
if  let Err(e) = self.tx_output.send(output).await {
tracing::warn!("Failed to output certificate: {e}");
}
}

// DAG 라운드 수 메트릭 업데이트
self.metrics.consensus_dag_rounds
.with_label_values(&[])
.set(state.dag.len() as i64);
},
```

```
// 위원회 변경 처리
result = self.rx_reconfigure.changed() => {
// ... 위원회 변경 로직
}
}
}
}
}
```

11.5.2 테스트에서 확인된 성능 특성

```
#[cfg(test)]
mod tests {
#[test]
fn state_limits_test() {
let gc_depth = 12;
let rounds: Round = rand::thread_rng().gen_range(10..100);

// "최적" 인증서로 테스트 (모든 노드가 정상 동작)
let (certificates, _next_parents) =
test_utils::make_optimal_certificates(&committee, 1..=rounds, &genesis, &keys);

for certificate in certificates {
```

```
tusk.process_certificate(&mut state, consensus_index, certificate).unwrap();
}
// 최적 조건에서 DAG 크기가 6 라운드 이하로 유지됨을 확인
let n = state.dag.len();
assert!(n <= 6, "DAG size: {}", n);
}

#[test]
fn imperfect_state_limits_test() {
// 33% 실패율로 테스트
let (certificates, _next_parents) =
test_utils::make_certificates(&committee, 1..=rounds, &genesis, &keys, 0.333);

for certificate in certificates {
tusk.process_certificate(&mut state, consensus_index, certificate).unwrap();
}

// 비최적 조건에서도 GC 깊이 이하로 유지됨을 확인
let n = state.dag.len();
assert!(n <= gc_depth as usize, "DAG size: {}", n);
}
}
```

11.6 실제 배포 고려사항

11.6.1 설정 관리

```
// 실제 벤치마크에서 사용되는 설정들
pub struct BenchmarkConfig {
pub faults: usize, // 0 (장애 노드 수)
pub committee_size: usize, // 4 (위원회 크기)
pub workers_per_node: usize, // 1 (노드당 워커 수)
pub input_rate: usize, // 50,000 tx/s
pub transaction_size: usize, // 512 B
pub header_size: usize, // 1,000 B
pub max_header_delay: Duration, // 100 ms
pub gc_depth: Round, // 50 rounds
pub batch_size: usize, // 500,000 B
pub max_batch_delay: Duration, // 100 ms
}
```

11.6.2 실제 성능 결과

```
벤치마크 결과 (4노드 로컬 테스트):
- 컨센서스 TPS: 46,478 tx/s
- 컨센서스 지연시간: 464 ms
- End-to-end TPS: 46,149 tx/s
```

```
- End-to-end 지연시간: 557 ms
```

11.7 코드 품질과 유지보수성

11.7.1 에러 처리 패턴

```
// 일관된 에러 타입 사용
pub type StoreResult<T> = Result<T, StoreError>;

// 구조화된 에러 정의
#[derive(Debug, Error)]
pub enum ValidatorDagError {
#[error("No remaining certificates in Dag for this authority: {0}")]
OutOfCertificates(PublicKey),
#[error("No known certificates for this authority: {0} at round {1}")]
NoCertificateForCoordinates(PublicKey, Round),
#[error("Dag invariant violation {0}")]
DagInvariantViolation(#[from] dag::node_dag::NodeDagError),
}
```

11.7.2 테스트 커버리지

```
// 포괄적인 테스트 케이스들
```

```rust
#[cfg(any(test))]
#[path = "tests/tusk_tests.rs"]
pub mod tusk_tests;

#[cfg(test)]
#[path = "tests/bullshark_tests.rs"]
pub mod bullshark_tests;

#[cfg(any(test))]
#[path = "tests/dag_tests.rs"]
pub mod dag_tests;
```

11.7.3 문서화와 트레이싱

```rust
// 상세한 트레이싱 지원
#[instrument(level = "info", skip_all)]
pub async fn new_from_store(/* ... */) -> Self {
// 구현...
}

// 조건부 로깅으로 성능 최적화
#[cfg(not(feature = "benchmark"))]
if output.consensus_index % 5_000 == 0 {
```

```
tracing::debug!("Committed {}", certificate.header);

}



#[cfg(feature = "benchmark")]

for  digest  in  certificate.header.payload.keys() {

tracing::info!("Committed {} -> {:?}", certificate.header, digest);

}
```

이러한 실제 구현 분석을 통해 Narwhal이 이론적 설계를 어떻게 실용적이고 효율적인 코드로 구현했는지 확인할 수 있습니다. 특히 성능 최적화, 메모리 관리, 에러 처리 등에서 실제 운영 환경을 고려한 세심한 구현이 돋보입니다.